[4조] WM-811K 기반 웨이퍼 불량 분류 모델 실험 보고서

P282 김민정 / P285 김지수 / P289 박규리 / P293 오승민

1. SUMMARY

결론

최종 모델은 **Baseline + Class Weight(CE 역빈도 가중)**로 선정했다. 소수 3개 클래스의 Recall은 **0.7957** → **0.9011**로 개선됐고, Macro-F1은 기준선과 큰 차이가 없었다. Accuracy는 다소 낮아졌지만, 이번 실험에서는 전체 정확도보다 불량을 놓치지 않는 것을 우선으로 했다.

실험 전 소수 3개 클래스의 Recall을 주 지표로 정하고, Macro-F1을 함께 확인하는 방식으로 모델을 비교했다. 8개 조합을 3개 seed로 반복한 결과, Class Weight를 적용했을 때 소수 클래스 Recall이 가장 크게 개선되면서 평가 기준도 만족해 최종안으로 선정했다.

주요 결과

1. **소수 클래스 검출 성능은 Class Weight를 적용했을 때 가장 크게 개선됐다.** Dropout, BatchNorm, Weight Decay를 적용했을 때는 성능 개선이 크지 않았고, 오히려 일부 클래스의 Recall이 낮아졌다.
2. **규제 기법만으로 과적합을 해결하기는 어려웠다.** Train과 Valid의 성능 차이는 일부 줄었지만, 그 과정에서 소수 클래스 성능까지 함께 떨어지는 경우가 있었다.
3. **규제를 여러 개 함께 적용했을 때는 성능이 더 크게 떨어지는 경우도 확인됐다.** 현재 구조에서는 규제를 추가하는 것보다 손실 함수에 클래스별 가중치를 주는 방법이 더 효과적으로 나타났다.

후속 과제

최종안에서도 Scratch의 Recall은 0.4132로 가장 낮다. Class Weight만으로는 이를 개선하는 데 한계가 있어, 후속 실험에서는 입력 해상도와 리사이즈 방식, 특징 추출 구조를 개선해 Scratch 검출 성능을 높일 예정이다.

2. 문제 정의 및 성능 판단 기준

2-1. 문제 정의 (Pain Point)

반도체 웨이퍼 검사에서 발생하는 오류는 크게 오탐(FP)과 미검출(FN)로 나눌 수 있다.
오탐(FP)은 정상 웨이퍼를 불량으로 판단하는 경우이고, 미검출(FN)은 불량 웨이퍼를 정상으로 판단하는 경우이다.

두 오류 중에서는 미검출(FN)의 비용 부담이 더 크다. 정상 웨이퍼를 불량으로 잘못 판단하면 엔지니어의 추가 확인을 통해 처리할 수 있지만, 불량 웨이퍼를 놓치면 그대로 후공정에 투입될 수 있다. 이후 불량이 발견될 경우 이미 투입된 가공 비용까지 손실로 이어질 수 있다.

따라서 이번 모델은 전체 Accuracy를 높이는 것보다 실제 불량을 얼마나 놓치지 않는지를 우선적으로 체크했다. 검사 데이터에서 정상 데이터의 비중이 높은 만큼 Accuracy만으로는 소수 불량 클래스의 검출 성능을 제대로 평가하기 어렵기 때문에, Recall과 Macro-F1을 함께 주요 성능 지표로 사용했다.

문제는 이 비용 구조가 Accuracy에 전혀 반영되지 않는다는 점이다. 검사 데이터는 본질적으로 정상이 압도적으로 많기 때문에, 모델이 모든 웨이퍼를 정상이라고 찍어도 높은 정확도가 나온다. 잡아야 할 불량을 잡지 못해도 숫자는 좋아 보인다. 실제로 데이터 불균형이 큰 환경에서는 Accuracy만으로 성능을 판단하기 어렵기 때문에 Precision과 Recall을 함께 봐야 한다는 연구 결과도 있다. (Akele & Gurau, 2026)

2-2. 데이터 특성

WM-811K는 실제 반도체 팹에서 수집된 **811,457장의 웨이퍼맵 데이터**로 구성되어 있다.
이번 실험에서는 이 데이터의 불균형과 라벨 특성을 고려해 모델을 평가했다.

(1) 클래스별 데이터 편차가 큼

불량 유형별 데이터 수 차이가 크다. 가장 많은 Edge-Ring은 **9,680장**인 반면 Near-full은 **149장**으로 약 65배 차이가 난다. 정상 데이터를 5,000장으로 줄인 Baseline에서도 불량 유형 간 데이터 편차는 그대로 남아 있다.

(2) 소수 클래스가 전체 Accuracy에 미치는 영향은 제한적

학습 데이터 19,532장 가운데 Near-full은 95장으로 전체의 0.49%에 불과하다. Near-full, Donut, Random을 모두 합쳐도 비중은 5.1% 수준이다.
따라서 이들 소수 클래스를 제대로 검출하지 못하더라도 전체 Accuracy에는 큰 차이가 나타나지 않을 수 있다. 이번 실험에서 Accuracy만으로 모델 성능을 판단하지 않은 이유도 여기에 있다.

(3) 정상 클래스 해석

따라서 `none`의 성능을 해석할 때는 정상 데이터와 라벨이 없는 데이터가 함께 포함되어 있다는 점을 고려할 필요가 있다. `none`의 Recall을 정상 웨이퍼의 검출 성능으로 그대로 해석하기는 어렵다.

2-3. 성능 판단 기준

구분	지표	정의 및 이유
주 지표	소수 3개 클래스 Recall (Near-full · Donut · Random)	앞서 정의한 비용 구조를 반영해, 실제 불량을 얼마나 놓치지 않았는지를 가장 중요하게 봤다. 따라서 세 소수 클래스의 Recall을 주 지표로 두고 미검출을 줄이는 것을 최우선 목표로 설정했다.
게이트	Macro-F1	소수 클래스 Recall만 높이는 과정에서 특정 클래스로 예측이 쏠리는 것을 방지하기 위한 기준이다. Recall 개선과 함께 전체적인 분류 성능도 유지할 수 있도록 기준선 대비 Macro-F1이 0.05 이상 낮아지는 경우는 후보에서 제외했다.
동등 판정	Macro-F1 표준편차 0.0164	동일한 설정에서 seed만 변경해 3회 반복한 결과, Macro-F1의 표준편차가 0.0164로 나타났다. 따라서 이 범위보다 작은 차이는 실질적인 성능 차이로 보지 않고, 성능이 비슷할 경우에는 구조가 더 단순한 모델을 우선했다.
참고	Overall Accuracy	정상 데이터가 많은 데이터 특성상 Accuracy는 다수 클래스의 영향을 크게 받는다. 따라서 모델 선정의 주요 기준으로 사용하지 않고, 개선 과정에서 전체적인 성능이 지나치게 떨어지지 않았는지를 확인하는 보조 지표로 활용했다.

*** 소수 클래스 정의 방법**

데이터 수가 가장 적은 **Near-full, Donut, Random**을 소수 클래스로 정의하고,
세 클래스의 Recall 평균을 주요 성능 지표로 사용했다.
실험 간 일관된 비교를 위해 대상 클래스는 실험 시작 시 고정했다.

2-4. 비교 프로토콜

모델 별 성능을 공정하게 비교하기 위해 **데이터 분할, 학습 조건, 평가 방식은 동일하게 유지하고 실험 기법만 변경하는 방식을 택했다.**

(1) 데이터 분할

전체 데이터는 `random_state=42`를 기준으로 Train : Valid : Test = 64 : 16 : 20으로 나눴다. 클래스별 데이터 편차를 고려해 모든 단계에서 stratified split을 적용했으며, 데이터가 적은 클래스도 각 데이터셋에 일정하게 포함되도록 했다.

(2) 학습 조건

모든 실험에 동일한 Early Stopping(max 30 epoch, patience 5, valid loss 기준)을 적용해 모델별 학습 횟수 차이에 따른 영향을 줄였다.

(3) 반복 측정

각 실험은 seed 42, 1, 7로 3회 반복해 평균과 표준편차를 확인했다. 특히 Near-full과 같이 데이터가 적은 클래스는 일부 샘플의 분류 결과에 따라 Recall이 크게 달라질 수 있어, 단일 실행 결과만으로 모델 성능을 판단하지 않았다.

(4) 고정 조건

모든 실험에서 모델 구조(Conv 32-64 + Flatten), 활성 함수(ReLU), Optimizer(Adam, lr=1e-3), batch size(64), 입력 크기(64×64, 1채널)를 동일하게 유지했다.

(5) 평가 방식

먼저 Valid 성능을 기준으로 실험 조합을 비교해 최종 모델을 선정했다. 이후 선정된 모델에 한해 Test 성능을 확인했으며, 모델 선정 과정에서는 Test 데이터를 사용하지 않았다.

---

3. 진단 — 데이터와 Baseline

3-1. 데이터 진단

라벨 정의의 한계

원본 WM-811K 811,457장 중 실제 불량 유형이 부여된 데이터는 **172,950장**이고, 나머지 **638,507장은 라벨이 비어 있다.** Baseline에서는 라벨이 없는 데이터를 모두 `none`으로 처리하고 있어, `none` 785,938장 중 실제 `none` 라벨은 **147,431장(18.8%)**에 불과하다. 나머지 81.2%는 실제 라벨이 확인되지 않은 데이터이다.

이번 실험에서는 Baseline과의 비교를 위해 이 라벨 정의를 그대로 사용했다. Train/Valid/Test에 동일한 기준을 적용했기 때문에 실험 간 상대적인 성능 비교에는 문제가 없다. 다만 `none`을 단순히 정상 웨이퍼로 보기는 어렵다. 실제로는 정상 데이터와 라벨이 없는 데이터가 함께 포함되어 있기 때문이다.

따라서 `none`의 성능을 **정상 웨이퍼에 대한 성능으로 그대로 보기는 어렵다.** 정상 데이터와 라벨이 없는 데이터가 함께 포함되어 있기 때문이다. `none`의 Precision과 Recall도 이 점을 감안해서 확인해야 한다.

불량 유형 간 불균형

라벨이 있는 8개 불량 유형의 데이터 수는 다음과 같다.

유형	개수	유형	개수
Edge-Ring	9,680	Scratch	1,193
Edge-Loc	5,189	Random	866
Center	4,294	Donut	555
Loc	3,593	Near-full	149

가장 많은 Edge-Ring(9,680)과 가장 적은 Near-full(149)은 약 65배 차이가 난다. Baseline에서는 데이터를 줄일 때 불량 데이터는 그대로 두고 `none`만 5,000장으로 샘플링한다. 그래서 전체 데이터 수를 줄인 뒤에도 불량 유형 간 65배 차이는 그대로 남는다.

`none`을 5,000장으로 줄이고 나면 Edge-Ring과 Edge-Loc은 오히려 `none`보다 데이터가 많아진다. **결국 `none`의 데이터만 줄었을 뿐, 불량 유형 간 데이터 차이는 그대로 남아 있다.**

불량 유형별 데이터 수 차이가 큰 만큼, 클래스별 차이를 손실 함수에 반영할 필요가 있다.

웨이퍼 맵 크기 불일치

라벨이 있는 웨이퍼 맵은 서로 다른 Shape가 **346종**이며, 세로 15~212, 가로 3~204로 크기 차이가 크다. 픽셀 값은 `0=배경`, `1=정상 다이`, `2=불량 다이` 세 가지로 구성된다.

Baseline에서는 모든 웨이퍼 맵을 nearest-neighbor 보간(`order=0`)으로 64×64 크기에 맞춘다. 작은 26×26 맵은 약 2.5배 확대되고, 200을 넘는 큰 맵은 약 3배 축소된다. 특히 큰 맵에 포함된 얇은 Scratch는 축소 과정에서 특징이 손실될 가능성이 있다.

3-2. 모델 구조 진단

Baseline(3 seed 평균)은 Train Accuracy 91.79%, Valid Accuracy 83.80%로 7.99%p의 성능 차이가 나타났다. 파라미터 구성을 확인한 결과, 특정 층에 파라미터가 집중되어 있어 과적합에 영향을 주는 구조적 요인을 확인할 수 있었다.

Layer	파라미터 수	비중
Conv1 (1→32)	320	0.04%
Conv2 (32→64)	18,496	2.25%
FC1 (12544→64)	**802,880**	**97.64%**
FC2 (64→9)	585	0.07%
계	**822,281**	100%

전체 파라미터의 97.6%가 Flatten 직후 FC1 한 층에 집중되어 있다. 14×14 특징맵을 12,544차원으로 펼친 뒤 대규모 FC층을 연결하는 현재 구조는 학습 데이터에 과도하게 맞춰질 가능성이 높다. 규제 적용 시 파라미터가 집중된 FC1을 우선 대상으로 검토할 필요가 있다.

클래스별 성능에서는 Scratch의 Recall이 가장 낮게 나타났다. Valid Recall은 0.1745로 191장 중 33장만 정확하게 검출했지만, 전체 Accuracy는 83.80%로 나타났다. 전체 Accuracy만으로는 소수 클래스의 낮은 검출 성능을 확인하기 어렵다.

---

4. 최적화 전략 및 실험 설계

4-1. 문제 원인 분석

현재 모델에서 다양한 개선점이 있지만, 크게 두 가지의 문제로 나뉜다. 첫번째는 과적합 문제이다. BaseModel 기준으로 현재 Train과 Valid의 Gap이 7.99%p로 과적합되었음을 시사한다. 두번째는 클래스 불균형으로 인한, 소수 클래스 분류 성능문제이다. 현재 소수 클래스 중 가장 성능이 안나오는 (Scratch Recall)을 확인하면 0.1745가 나온다.

	지표	문제 원인	대응 방법
문제 A	Train–Valid gap 7.99%p	FC1에 파라미터 97.6% 집중, 위치를 암기하는 방식으로 학습됨(OverFitting)	**규제 추가**
문제 B	Scratch Recall 0.1745 Recall 평균 0.8380	클래스 유형간 최대 65배의 개수 격차가 존재함	**클래스 불균형 보정**

두 문제는 원인이 다르므로 서로 방법의 해결책을 시도해야 한다. 규제를 아무리 걸어도 클래스 간 격차는 해소되 지 않고, 클래스 가중치를 수정하는 방식을 활용해도 과적합 문제는 크게 해결되지 않는다. 따라서 과적합 문제 해결과 클래스 **불균형 보정**을 두 축으로 두고, 각각 효과를 확인한 뒤, 조합하여 결과를 확인한다.

앞선 분석대로면, 모델 구조 변경도 시도해 볼 수 있는 방법이다. Conv 채널을 확장해서 더 다양한 정보를 반영하도록 시도할 수 있다. 그러나 구조를 바꾸면 파라미터 수와 학습 과정에 변화가 생겨서 규제와 클래스 불균형 보정의 순수한 효과를 분리 측정할 수 없다. 따라서 논의 끝에 Baseline 구조를 고정한 상태에서 규제와 클래스 불균형 보정을 통해 성능 향상을 시도해보기로 결정했다. 모델 구조 변경은 후속 과제로 둔다.

4-2. 문제 해결 전략

(1) 문제 A(과적합) → 규제 추가

기법	원리	적용 위치
Early Stopping	Valid loss가 patience epoch 동안 개선되지 않으면 중단하여, 모델이 학습 데이터를 외우기 시작하는 시점 이전의 가중치를 보존	학습 루프 (**전 실험 공통**)
Dropout (p=0.3)	뉴런을 확률 p로 무작위 비활성화하여 특정 뉴런 조합에 대한 공적응(co-adaptation)을 차단	**FC1 뒤** — 파라미터 97.6%가 집중된 지점
BatchNorm	배치 단위로 활성 분포를 정규화하여 학습을 안정화하고, 배치 표본에서 오는 잡음이 약한 규제로도 작용	각 Conv 직후
Weight Decay (1e-4)	가중치 L2 norm에 페널티를 부과해 결정 경계를 완만하게 유지	Optimizer (Adam)

(2) 문제 B(클래스 불균형) → 손실 함수 보정

먼저 이 문제를 해결하기 위해 Data 처리를 고려해 볼 수 있다. 다음과 같은 방법을 시도해 볼 수 있다. 웨이퍼 데이터이므로 Data Augmentation - Rotation/Flip등을 고려할 수 있으며, 현재 학습 데이터에서 버려진 Labeling 되지 않은 데이터를 강화 학습 모델, 직접 라벨링을 통해 데이터를 추가할 수 있다. 그러나 이번 모델 구축에서는 벗어나는 범위이므로 손실 함수 안에서 클래스 비중을 재정의한다. **Class Weight**는 클래스별 가중치를 CrossEntropy에 부여해 소수 클래스 오분류에 더 큰 패널티를 부여하는 것이다. 실제로 손실에 반영되는 값은 다음과 같다.

클래스	Train 건수	가중치	클래스	Train 건수	가중치
Near-full	95	**22.84**	Loc	2,299	0.94
Donut	355	6.11	Center	2,748	0.79
Random	555	3.91	none	3,200	0.68
Scratch	764	2.84	Edge-Loc	3,321	0.65
			Edge-Ring	6,195	**0.35**

이렇게 설정하면 Near-full을 틀리는 패널티가 Edge-Ring을 틀렸을 데에 비해 **65배가 된**다.

(3) 실험 설계 원칙

- **단일 변인 통제**
  - 각 기법의 순수 효과를 확인하기 위해 #2~#5는 Baseline에서 한 항목만 바꾼다.
- **단독 → 조합**
  - #6~#8은 단독 결과를 확인한 뒤 누적 조합한다. 조합에서 성능이 떨어지면 두 기법이 상충한 것으로 해석한다.
- **고정 조건**
  - 전 실험에서 구조(Conv 32-64 + Flatten), ReLU, Adam(lr=1e-3), batch_size=64, seed=42/1/7, 동일 split을 유지한다.
  - Seed에 따른 잡음을 최소화하기 위해 여러번 실험을 진행한다
  - Early Stopping은 Base 모델에 기본적으로 적용한다

(4) 실험 목록

#	실험	변경점	해결 문제	검증하려는 가설
1	Base 모델	— (Early Stopping만)	기준 모델	비교 기준점 및 seed 잡음 폭 측정
2	BatchNorm	norm=batch	과적합	학습 안정화가 gap과 소수 Recall에 함께 도움이 되는가
3	Dropout	p=0.3 (FC1 뒤)	과적합	파라미터가 몰린 지점을 규제하면 gap이 줄고 Recall은 유지되는가
4	Class Weight	CE 역빈도 가중치	클래스 불균형	소수 Recall이 오르는가
5	Weight Decay	Adam wd=1e-4	과적합	L2 단독 효과
6	Dropout + BN	p=0.3, norm=batch	과적합	두 규제가 상호 보완적인가
7	Dropout + BN + CW	p=0.3, norm=batch,CE 역빈도 가중치	복합	두가지 다른 방향의 해결 책을 더했을 때 각 효과가 발휘 되는가?
8	Dropout + BN + CW + WD	전체	복합	모든 해결방안을 추가했을 때 이득이 있는가?

4-3. 실험 판단 기준

#	기법	예상 방향	성공 판정 기준	실패 시 해석
2	BatchNorm	수렴 안정화	gap 감소, 소수 Recall 유지	배치에 소수 클래스가 거의 안 들어와 통계가 다수 클래스만 반영
3	Dropout	gap 축소	gap 7.99%p 대비 감소, 소수 Recall 유지	Valid Acc 동반 하락 = 과규제. p를 낮춰 재실험
4	Class Weight	소수 Recall 상승	**소수 Recall 0.8380 초과**, Macro-F1 하락 0.05 이내	Recall만 오르고 Macro-F1이 크게 떨어지면 애매한 샘플을 소수 클래스로 남발한 것
5	Weight Decay	gap 소폭 축소	gap 감소, 타 지표 불변	L2를 gradient에 더하는 결합형이라 효과가 희석될 수 있음. "WD 무용"이 아니라 "AdamW 재실험 필요"로 해석
6·7·8	조합	가산적 개선	각 단독 실험 대비 두 축 모두에서 개선	단독보다 나빠지면 두 기법이 상충한 것

4-4. 최종 모델 선정 규칙

최종 모델은 실험 전에 코드로 고정해서 인위적인 개입을 없앤다

1. 조건을 통과한 후보 모델만 자격을 얻는다 — Macro-F1 ≥ 기준선 − 0.05 **그리고** 소수 Recall ≥ 기준선
2. 통과 후보 중 **Recall(소수 3개 클래스)가 가장 높은 것**을 고른다
3. 주 평가 지표 차이가 잡음 (0.0164) 이내면 동등한 모델로 보고, 더 단순한 구성의 모델을 남긴다.
4. 선정된 최종 1개의 모델만 테스트 데이터로 평가한다.

---

5. 실험 결과

5-1. 실험 상세 (Valid, 3 seed 평균)

#	실험 개요	주요 하이퍼파라미터	Train-Acc	Valid-Acc	Valid-Macro-F1	Valid-Recall (소수)	비고 (best epoch)
1	**Base 모델**	Early Stopping, patience=5	91.79%	83.80%	0.7655 ± 0.0164	0.8380	5 · 기준선 · gap 7.99%p
2	BatchNorm	Conv 직후	89.60%	83.99%	0.7520 ± 0.0216	0.7868	4 · gap 5.60%p · Recall 불만족
3	Dropout	p=0.3	91.56%	**84.69%**	0.7497 ± 0.0090	0.7953	8 · gap 6.88%p · Recall 불만족
4	**Class Weight**	CE 역빈도 가중	85.99%	78.72%	0.7489 ± 0.0199	**0.9311**	5 · gap 7.27%p · **최종 선정**

#	실험 개요	주요 하이퍼파라미터	Train-Acc	Valid-Acc	Valid-Macro-F1	Valid-Recall (소수)	비고 (best epoch)
5	Weight Decay	Adam wd=1e-4	92.33%	84.55%	**0.7704 ± 0.0222**	0.8328	5 · gap 7.78%p · Recall 불만족
6	Dropout + BN	p=0.3, BN	88.55%	82.84%	0.6432 ± 0.0450	0.5364	10 · **8개 중 최하위** · 모든 조건 불만족
7	Dropout + BN + CW	p=0.3, BN, 역빈도	77.86%	73.73%	0.6572 ± 0.0266	0.9338	15 · gap 4.13%p · Macro-F1 불만족
8	Dropout + BN + CW + WD	• wd=1e-4	81.95%	75.72%	0.6744 ± 0.0227	**0.9380**	16 · 소수 Recall 최고 · Macro-F1 불만족

> **조건 통과: #1, #4 뿐.** Macro-F1 하한 0.7155 (= 0.7655 − 0.05), 소수 Recall 하한 0.8380.
> #2·#3·#5는 소수 Recall이 기준선 아래, #6·#7·#8은 Macro-F1이 하한 아래.

5-2. 클래스별 Valid Recall 상세

클래스	Train 건수	Valid 건수	#1 Base	#2 BN	#3 Drop	#4 CW	#5 WD	#6 D+BN	#7 +CW	#8 +WD
Edge-Ring	6,195	1,549	0.9604	0.9862	0.9761	0.9621	0.9688	0.9697	0.9475	0.9606
Edge-Loc	3,321	830	0.7968	0.7582	0.8141	0.7108	0.8104	0.8386	0.5988	0.7060
none	3,200	800	0.8025	0.8338	0.7883	0.6329	0.8100	0.8225	0.6333	0.6121
Center	2,748	687	0.9544	0.9287	0.9738	0.9243	0.9500	0.9306	0.9112	0.9112
Loc	2,299	575	0.7067	0.7014	0.7565	0.5414	0.7159	0.6771	0.3971	0.3299
Scratch	764	191	0.1745	0.1518	0.0401	**0.4154**	0.1850	0.0035	0.2443	0.4538
Random (소수)	555	138	0.8285	0.8768	0.8889	0.9372	0.8357	0.6860	0.8913	0.9179
Donut (소수)	355	89	0.7828	0.7753	0.7191	0.8839	0.8015	0.6592	0.9101	0.9101
Near-full (소수)	95	24	0.9028	0.7083	0.7778	0.9722	0.8611	**0.2639**	1.0000	0.9861
합계	**19,532**	**4,883**	—	—	—	—	—	—	—	—

6. 실험 결과 분석

6-1. 요약

첫째, 규제를 통한 과적합 해결은 실패했다. Baseline의 train-valid gap인 7.99%p를 단독 규제를 통해 줄인 최대치는 BatchNorm을 적용했을 때 5.60%p인데, 그마저도 Macro-F1과 Recall이 함께 감소했다. 즉 약간의 일반화 성능을 챙기는 대가로, 우리 모델의 핵심인 Recall을 잃었다.

둘째, 클래스 불균형 문제를 해결하는 것은 규제가 아닌 손실 함수 가중치였다. Recall 상위 지표 3개를 Class Weight를 넣은 실험이 달성했다. Class Weight를 넣지 않은 5개의 실험은 모두 기준선을 달성하지 못했다.

6-2. 과적합 해결 실패

기법	gap	Baseline 대비	Macro-F1	소수 Recall	판정
Baseline (#1)	7.99%p	—	0.7655	0.8380	기준
BatchNorm (#2)	5.60%p	−2.39%p	0.7520	0.7868	gap은 줄었으나 두 지표 모두 하락
Dropout 0.3 (#3)	6.88%p	−1.11%p	0.7497	0.7953	소수 Recall 하락
Weight Decay (#5)	7.78%p	−0.21%p	**0.7704**	0.8328	효과 없음(잡음과 구분불가)

BatchNorm은 gap을 가장 많이 줄였지만 대가가 컸다. Valid Accuracy는 83.99%로 Baseline보다 높았는데, **Near-full Recall이 0.9028 → 0.7083으로 떨어졌다.** 배치 통계로 정규화하는 방식이 소수 클래스에 불리하다는 점을 시사한다.

Dropout p=0.3은 Valid Accuracy 84.69%로 8개 중 1위였다. 그러나 Macro-F1은 0.7497로 오히려 내려갔고 **Scratch Recall이 0.1745 → 0.0401로 감소하여, 소수 클래스를 검출할 수 없는 수준이 됐다.** 전체 정확도만 보면 성공, 클래스별로 보면 실패인 전형적인 사례다. 앞서, Accuracy를 참고 지표로만 두었는데, 이 사례에서도 확인할 수 있었다. Accuracy는 높아졌지만, 이는 애매한걸 전부 "문제 없음"으로 분류해버려서 상승하는 것이다.

Weight Decay는 gap을 0.21%p밖에 줄이지 못했다. Macro-F1 0.7704는 8개 중 최고지만 Baseline 대비 +0.0049로 **잡음 폭 0.0164 안에 있어 개선이라 부를 수 없다.** Weight Decay 설정이 L2를 gradient에 직접 더하는 결합형이라 Adam의 작동 방식과 상충해 의도한 규제가 희석된 것으로 보인다. **"Weight Decay가 무용하다"가 아니라 "Adam의 L2 구현에서는 효과를 확인할 수 없었다"가 정확한 결론이며, AdamW 재실험이 필요하다.**

결론: 규제를 통한 과적합 해결이 전반적으로 실패한 것은 앞선 내용과 거의 일치한다. 파라미터의 97.6%가 FC1 한 층에 몰린 구조에서는, 그 층 뒤에 Dropout을 붙이는 것만으로 편중 자체를 해소할 수 없다. **과적합을 실제로 줄이려면 규제 추가가 아니라 구조 변경이 필요하다는 결론에 도달한다.**

6-3. Class Weight를 통한 소수 클래스 검출력 향상

8개 실험을 Class Weight 적용 여부로 나누면 효과가 명확하다.

Class Weight	실험	소수 Recall 범위
적용 (3개)	#8, #7, #4	**0.9311 ~ 0.9380**
미적용 (5개)	#1, #5, #3, #2, #6	0.5364 ~ 0.8380

소수 Recall 상위 3위를 Class Weight 실험이 독점했고, 미적용 실험 중 최고인 Baseline(0.8380)조차 CW 최하위인 #4(0.9311)에 크게 못 미쳤다. 두 집단은 겹치지 않는다. 손실 함수에 클래스 가중치를 넣었는지 여부가 소수 클래스 검출력을 가르는 유일한 결정 변수였으며, 규제 기법의 조합이나 강도는 여기에 거의 영향을 주지 못했다.

가장 극적인 변화는 표본이 가장 적은 두 클래스에서 나타난다.

클래스	Valid 건수	Baseline (#1)	CW 단독 (#4)	변화
Near-full	24	0.9028	**0.9722**	+0.0694
Donut	89	0.7828	**0.8839**	+0.1011
Random	138	0.8285	**0.9372**	+0.1087
Scratch	191	0.1745	**0.4154**	**+0.2409 (×2.4)**

Baseline은 Scratch 191장 중 33장만 잡았지만 CW 적용 후 79장까지 올라간다.

그러나 대가도 명확하다. Class Weight는 공짜가 아니라 **다수·중간 크기 클래스를 희생시켜** 소수 클래스를 살렸다.

클래스	Train 비율	Baseline	CW 단독 (#4)	변화
none	16.38%	0.8025	0.6329	**−0.1696**
Loc	11.77%	0.7067	0.5414	**−0.1653**
Edge-Loc	17.00%	0.7968	0.7108	−0.0860

Valid Accuracy가 83.80% → 78.72%로 5.08%p 떨어진 것이 이 교환의 총합이다. 앞서 언급한대로 **이 하락은 미검출을 줄이기 위한 교환이며 실패라고 정의할 수는 없다.** 다만 Macro-F1이 0.7489로 −0.0166 하락한 것은 게이트(−0.05) 안이지만 잡음 폭(0.0164)과 거의 같은 크기라, Macro-F1은 유지됐다고 말하는 것이 정확하다.

6-4. 규제 조합 시 역효과 발생 : Dropout + BN

실험 #6의 결과가 가장 예상하지 못했던 점이다.

	Macro-F1	소수 Recall	Near-full	Scratch
Dropout 0.3 단독 (#3)	0.7497	0.7953	0.7778	0.0401
BatchNorm 단독 (#2)	0.7520	0.7868	0.7083	0.1518
Dropout + BN (#6)	**0.6432**	**0.5364**	**0.2639**	**0.0035**

두 단독 실험 모두 Baseline과 비슷한 수준이었는데, 합치자 **8개 실험 중 최하위**로 떨어졌다. Near-full은 24장 중 6장, Scratch는 191장 중 1장만 잡는 모델이 됐다. 4-3에서 "단독보다 나빠지면 두 기법이 상충한 것"이라 했는데 정확히 이에 해당한다. seed 표준편차도 0.0450으로 8개 중 가장 커서, 학습 자체가 불안정했음을 보여준다.

원인을 추정해보면 BatchNorm은 배치 단위 통계로 정규화하는데, 현재 설정인 batch_size=64에서 Near-full이 배치에 포함될 기댓값은 **0.31장**(0.49% × 64)이다. 세 배치에 한 장꼴이므로 BatchNorm이 계산하는 평균·분산은 사실상 다수 클래스만 반영하며, 소수 클래스 샘플은 자기와 무관한 통계로 정규화된다. 여기에 Dropout까지 걸리면 이미 왜곡된 소수 클래스의 정보가 이중으로 소실된다. Early Stopping의 최적 epoch가 10까지 늘어난 것도 학습이 안정적으로 수렴하지 못했음을 뒷받침한다.

6-5. 왜 #7·#8이 아니라 #4인가

주 평가 지표(소수 클래스의 Recall)만 보면 #8(0.9380) > #7(0.9338) > #4(0.9311) 순이다. 그런데 최소 만족 조건을 적용하면 #7·#8이 탈락한다.

	Recall(소수 클래스)	Macro-F1	Valid Acc	판정
기준선 (#1)	0.8380	0.7655	83.80	—
#4 Class Weight	0.9311	**0.7489** (만족)	78.72	**통과**
#7 D+BN+CW	0.9338	0.6572 (불만족)	73.73	탈락 (하한 0.7155)
#8 D+BN+CW+WD	**0.9380**	0.6744 (불만족)	75.72	탈락 (하한 0.7155)

#8이 #4보다 얻은 Recall(소수 클래스)**은 +0.0069다. 잡음 폭 0.0164의 절반도 안 되므로 seed를 바꾸면 뒤집힐 차이다.** 반면 잃은 것은 명확하다.

- **Macro-F1 0.7489 → 0.6744** (−0.0745). 이는 기준 하한 0.7155를 크게 밑도는 수치이다.
- **정상 Recall 0.6329 → 0.6121.** 정상 웨이퍼 800장 중 310장을 결함으로 오판한다. 검사 라인에 걸면 39%가 재검사 대상이 되어 실사용이 어렵다. 이는 애매한 것을 전부 결함으로 판단한 것이다. 미탐을 오탐보다 중요한 기준으로 설정은 했지만, 그렇다고 오탐을 아예 신경쓰지 않아도 되는 것은 아니다.
- **Loc 클래스 Recall 0.5414 → 0.3299.** 575장 중 190장만 잡는다.

즉 #7·#8의 소수 Recall 우위는 **다른 클래스를 대량으로 포기해서 얻은 것**이며, 그 손실이 6-4에서 확인한 Dropout+BN의 상충 문제를 그대로 보여주고 있다. **가장 단순한 #4가 거의 같은 Recall(소수 클래스)을 훨씬 적은 비용으로 달성한다.**

6-6. 최종 모델 선정과 Test 결과

최종 구성: Baseline 구조(Conv 32-64 + Flatten, ReLU) + Adam(lr=1e-3) + CrossEntropy(역빈도 가중) + Early Stopping(patience=5)

항목	내용/값
실험 #	#4
적용 기법 조합	Baseline + Class Weight (+ Early Stopping)
Test-Accuracy	**0.7780** ± 0.0212
Test-Macro-F1	**0.7321** ± 0.0131
Test-Recall (mean)	**0.9011** ± 0.0380 (소수 3클래스)

Baseline 대비 Test 성능

지표	Baseline (#1)	최종 (#4)	변화
소수 3클래스 Recall (주)	0.7957 ± 0.0464	**0.9011 ± 0.0380**	**+0.1054**
Macro-F1	0.7391 ± 0.0138	0.7321 ± 0.0131	−0.0070 (잡음 폭 이내)
불량 검출률	0.9414	**0.9501**	+0.0087
Accuracy (참고용)	82.60 ± 0.61	77.80 ± 2.12	−4.80%p

클래스별 Test Recall

클래스	Test 건수	Baseline	최종 (#4)	변화
Edge-Ring	1,936	0.9513	0.9516	+0.0003
Edge-Loc	1,038	0.7845	0.6962	−0.0883
none	1,000	0.7973	0.6320	**−0.1653**
Center	859	0.9499	0.9418	−0.0081
Loc	719	0.6565	0.5021	−0.1544
Scratch	238	**0.1835**	**0.4132**	**+0.2297 (×2.3)**
Random (소수)	173	0.8728	**0.9403**	+0.0675
Donut (소수)	111	0.7477	**0.8408**	+0.0931
Near-full (소수)	30	0.7667	**0.9222**	+0.1555
합계	**6,104**	—	—	—

실험을 통해 관찰한 트레이드오프가 테스트에서도 그대로 재현됐다. **소수 3개 클래스와 Scratch가 오르고, 정상·Loc·Edge-Loc이 내려갔다.** 구체적으로는 Scratch 238장 중 44장만 잡던 모델이 98장을 잡게 되었고, Near-full은 30장 중 23장에서 28장으로 늘었다. 대신 정상 웨이퍼 1,000장 중 368장이 결함으로 오판된다. 이는 기존보다 16% 증가한 수치이다.

---

7. 한계 및 후속 과제

이번 실험을 통해 소수 클래스의 미검출 문제는 일부 개선하였지만, 과적합과 Scratch 검출 성능 등은 충분히 해결하지 못했다. 또한 실제 라인에 적용을 고려했을 때, 오탐에 따른 재검사 비용과 데이터 분포의 차이도 추가로 검토할 필요가 있다.

(1) 과적합 개선에서의 한계

Dropout, BatchNorm, Weight Decay등 여러 규제 방법을 적용했지만 Train과 Valid 간 성능 차이는 크게 줄지 않았다. Baseline에서 약 8%p였던 gap이 최종안에서도 약 7%p 수준으로 남아 있어, 규제 기법만으로는 과적합을 충분히 줄이기 어려운 것으로 보인다.

현재 구조는 전체 파라미터의 대부분이 Flatten 이후 FC1에 집중되어 있어, 모델 구조 자체가 과적합에 영향을 주고 있을 가능성이 높다.

> **후속:** Flatten을 Global Average Pooling(GAP)으로 교체하고 Conv 채널을 32-64-128로 확장해, 파라미터를 FC층보다 특징 추출 영역에 배분하는 방향으로 추가 실험할 예정이다.

(2) 검출 성능이 낮은 Scratch

Class Weight 적용 후 Scratch의 Test Recall은 0.18 → 0.41로 개선됐지만, 전체 클래스 중에서는 여전히 가장 낮은 수준을 보였다.

학습 데이터가 더 적은 Near-full의 Recall이 높게 나타난 점을 고려했을 때, 이는 단순한 데이터 부족만으로 설명하기는 어렵다. Scratch가 가느다란 선 형태의 결함이라는 점에서, 원본 이미지를 64×64로 축소하는 과정에서 특징이 충분히 보존되지 않았을 가능성이 제기된다.

> **후속:** 입력 해상도를 128×128 이상으로 높여 특징 손실을 줄이는 실험을 진행할 예정이다. 다만 해상도를 높이면 Flatten 구조의 파라미터가 크게 증가하기 때문에, GAP 적용과 함께 검토할 필요가 있다.

(3) 오탐 증가에 따른 운영 부담

최종안에서는 정상으로 분류되는 Recall이 0.63 수준으로 나타나, 정상에 해당하는 웨이퍼를 결함으로 판단해 추가 확인이 필요한 경우가 적지 않았다.

이번 실험에서는 FN의 비용이 FP보다 크다는 전제하에 소수 클래스 Recall을 우선했지만, 실제 라인에서는 재검사 물량 자체가 운영 부담으로 이어질 수 있다. 그렇기 때문에 FN을 줄이는 것만으로 모델의 최적성을 판단하기에는 한계가 있다.

> **후속:** 현장 기준으로 FN과 FP의 비용 차이를 확인하고, 이를 반영한 기대 비용을 기준으로 예측 임계값과 모델 성능을 함께 비교할 예정이다.

(4) FN 유형별 구분 한계

집계한 것은 클래스별 Recall뿐이어서, "결함을 놓쳐 정상으로 분류한 경우"와 "다른 결함 유형으로 잘못 분류한 경우"가 구분되지 않는다. 후자는 재검사에서 걸러지므로 비용 구조가 전혀 다르다. 불량 검출률(0.9501)로 일부 보완했으나 유형 간 혼동은 여전히 섞여 있다.

> **후속:** 혼동행렬을 기준으로 결함 → 정상으로 분류된 경우를 별도로 집계해 실제 미검출 규모를 확인할 예정이다.

(5) 평가 지표에 따른 선정 결과 차이

이번 실험에서는 FN을 줄이는 것을 우선한다는 비용 구조에 따라 소수 클래스 Recall을 주 평가 지표로 설정했다.
Macro-F1을 기준으로 보았을 때 Baseline보다 뚜렷하게 개선된 모델은 없었다.

따라서 이번 실험에서 Class Weight가 가장 효과적인 방법이라는 결론은 FN을 더 중요하게 보는 현재의 평가 기준을 전제로 한 결과이다. FP와 FN의 비용이 달라지면 최종 모델도 달라질 수 있다.

(6) 학습 데이터와 실제 라인 간 분포 차이

Data Reduce를 통해 학습 데이터의 정상 비율을 약 16%까지 낮췄지만, 실제 라인에서는 정상이 약 97%를 차지한다.

학습 환경과 실제 환경의 데이터 분포가 크게 다른 만큼, 실제 적용 시 오탐이 현재 실험 결과보다 증가할 가능성이 있다. 따라서 이번 실험에서 확인한 성능을 실제 운영 성능으로 그대로 해석하기에는 한계가 있다.

> **후속:** 실제 라인의 데이터 분포를 반영한 검증셋을 별도로 구성하거나, 운영 환경에 맞춰 예측 임계값을 조정하는 방법을 검토할 예정이다.

---

9. 참고 문헌

> Akele, Y. K., & Gurau, V. (2026). A critical survey of machine learning-powered vision inspection in semiconductors manufacturing. *Array*, 100888. https://doi.org/10.1016/j.array.2026.100888

> Wu, M. J., Jang, J. S. R., & Chen, J. L. (2015). Wafer map failure pattern recognition and similarity ranking for large-scale data sets. *IEEE Transactions on Semiconductor Manufacturing*, 28(2), 154–164.

> WM-811K Baseline 참고 구현: Kaggle - WM-811k Wafermap